

TDVI: Inteligencia Artificial

Variational autoencoders (VAEs), Generative adversarial networks (GANs)

Licenciatura en Tecnología Digital

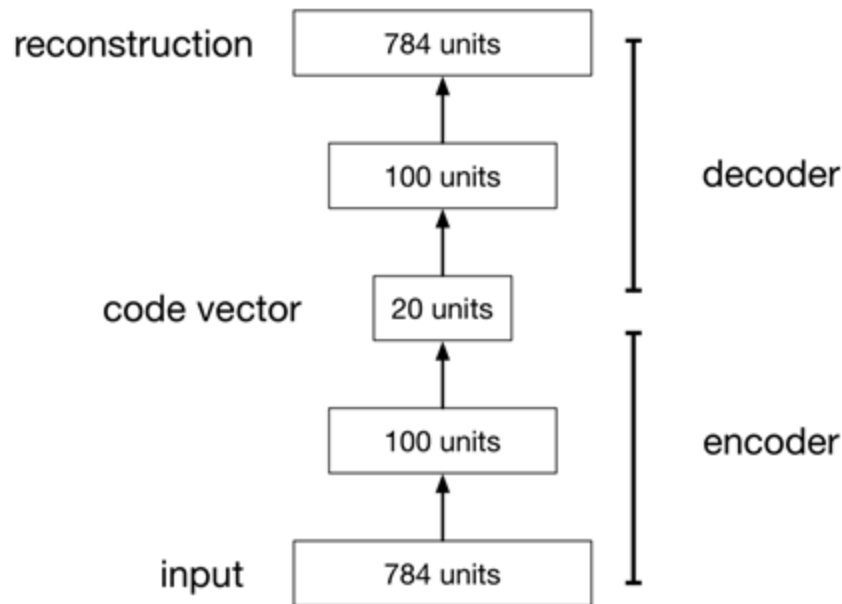


Agenda

1. **AutoEncoder**
2. Variational Autoencoders
3. Generative Adversarial Networks
4. Bibliografía

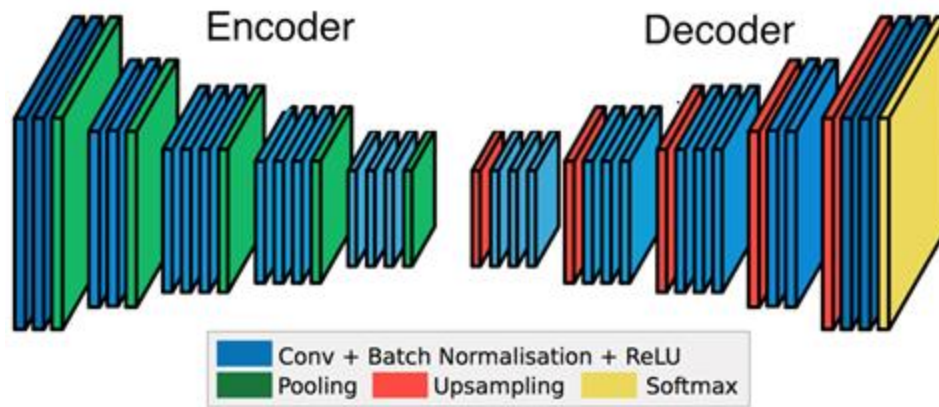
Autoencoder

- Un autoencoder es una red (feedforward en lo más básico) que recibe como entrada x y como salida retorna x
- Para que no sea trivial la red contiene un cuello de botella (bottleneck)
 - llevamos los datos a una dimensión mucho más pequeña que la entrada original

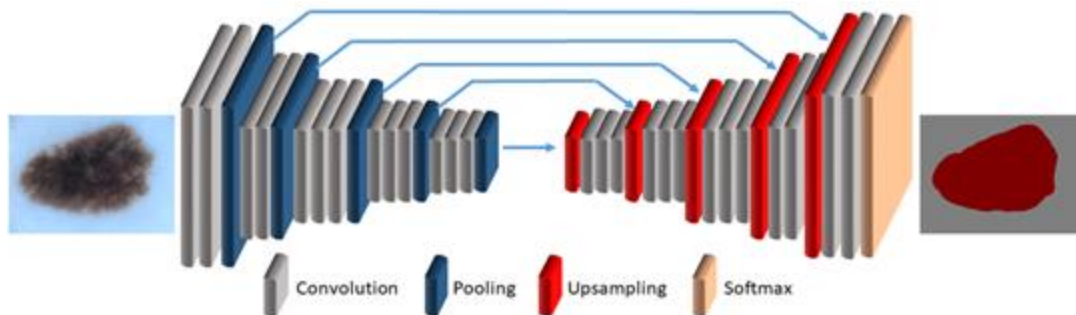
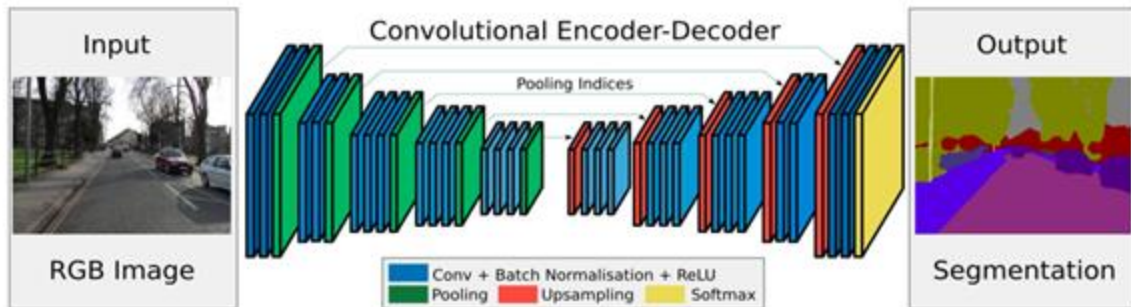


Encoder–Decoder

- **Encoder:** Convierte los datos de entrada en algo compacto y abstracto.
 - Una representación vectorial que “vive” en el espacio latente de la red neuronal.
- **Decoder:** Convierte el vector nuevamente al espacio original.



Segmentación

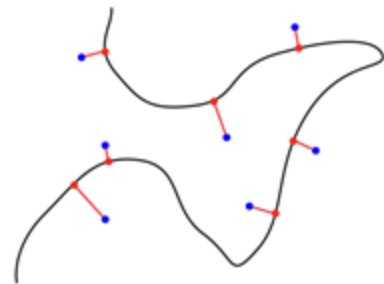
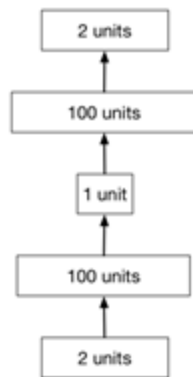


¿Por qué?

- Mapeamos datos de alta dimensionalidad en dos dimensiones:
 - visualización
- Compresión
- Aprender características abstractas (unsupervised) para luego utilizarlas en tareas supervisadas.
 - Los datos no etiquetados suelen abundar más que los datos etiquetados
- Aprender representaciones semánticamente significativas de ser posible:
 - Ej: interpolar entre imágenes

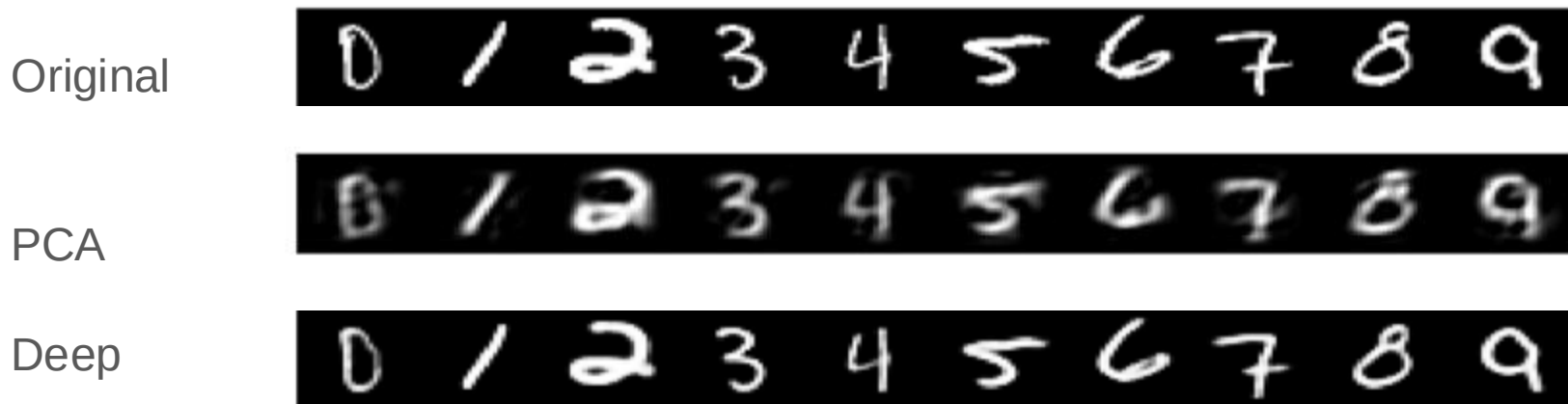
Deep Autoencoders

- Los autoencoders no lineales profundos aprenden a proyectar los datos, no en un subespacio, sino en un manifold no lineal.
- El manifold es la imagen del decodificador.
- Esto es una especie de reducción no lineal de dimensionalidad.



Deep Autoencoders

Los autoencoders no lineales pueden aprender códigos más potentes para una dimensionalidad dada, en comparación con los autoencoders lineales (PCA).



Limitaciones de autoencoders

- No son generativos
 - no definen una distribución
 - diseñada para reconstruir los datos de entrada a partir de ese punto.
 - No se tiene en cuenta la incertidumbre
 - la generación de datos novedosos no está inherentemente respaldada por la arquitectura
- ¿Cómo elegimos la dimensión del espacio latente

Agenda

1. AutoEncoder
2. **Variational Autoencoders**
3. Generative Adversarial Networks
4. Bibliografía

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Abstract

How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions are two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per datapoint, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

Demo

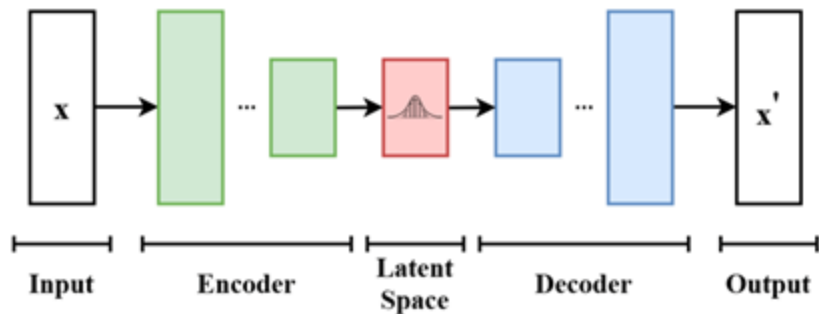
http://dpkingma.com/sgvb_mnist_demo/demo.html

Autoencoder variacional (VAE)

- Introduce una variante probabilística al asignar distribuciones de probabilidad a la representación latente.
 - en lugar de tener un punto específico en el espacio latente, se tiene una distribución de probabilidad sobre ese espacio.
 - Esto permite muestrear diferentes puntos en la distribución
- Incorpora un término de regularización que fuerza la representación latente a seguir una distribución específica (típicamente una distribución normal)
 - incentiva al modelo a aprender una estructura en el espacio latente

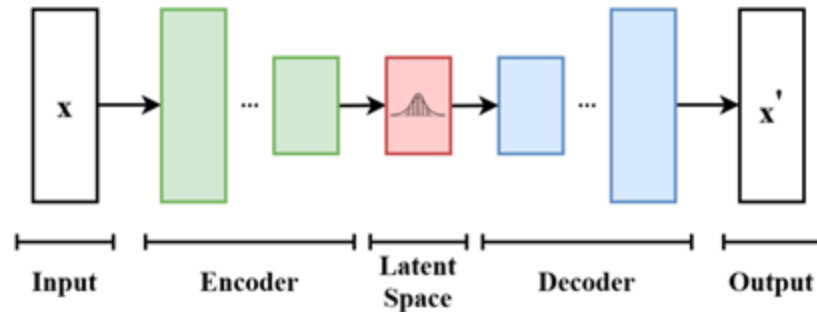
Autoencoder probabilístico

1. $f(h|x, w_f)$ es un codificador probabilístico,
 - a. dado un punto de datos x , produce una distribución (Ej: gaussiana) sobre los posibles valores del código h que podrían generar el punto de datos x .
2. $g(x|h, w_g)$ es un decodificador probabilístico
 - a. dado un código h , produce una distribución sobre los posibles valores correspondientes de x .



Autoencoder probabilístico

3. $g(x|h, w_g)$ el decoder puede ser tratado como un modelo generativo
- Samplea h de $P(h)$
 - Samplea x de $P(x | h, w_g)$



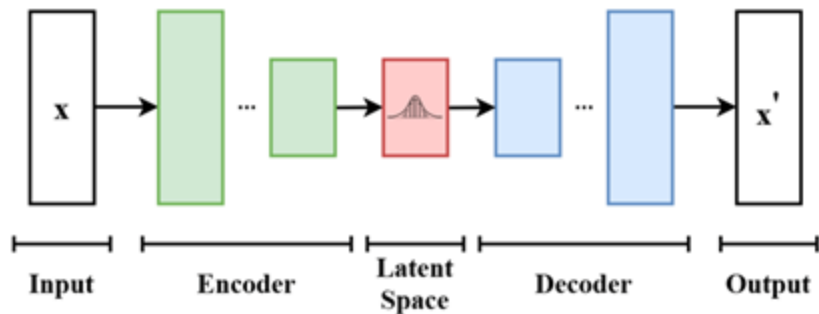
Autoencoder probabilístico

3. $g(x|h, w_g)$ el decoder puede ser tratado como un modelo generativo

- Samplea z de $P(h)$
- Samplea x de $P(x | h, w_g)$

¿Cómo elegimos $P(h)$?

No podemos utilizar $P(h | x, W_f)$ ya que es lo que estamos intentando generar



Variational Autoencoders

- Idea: Entrenar $P(h | x, w_f)$ para que se parezca a una distribución fija $N(h, 0, 1)$
→ Podemos setear $P(h)$ a $N(h, 0, 1)$

Función de costo:

$$\max \sum \log \Pr(x_n, W_f, W_n) - \underbrace{\text{KL}(\Pr(h | x_n; W_f) \| N(h, 0, t))}_{\text{Kullback-Leibler divergence Distance measure for distributions}}$$

Kullback-Leibler divergence Distance measure for distributions

VAE likelihood

$$\Pr(\mathbf{x}_n; \mathbf{W}_f, \mathbf{W}_g) = \int_{\mathbf{h}} \Pr(\mathbf{x}_n | \mathbf{h}; \mathbf{W}_g) \Pr(\mathbf{h} | \mathbf{x}_n; \mathbf{W}_f) d\mathbf{h}$$

- $P(\mathbf{h}|\mathbf{x}, \mathbf{w}_f)$ se tiene que parecer a $N(\mathbf{h}; 0, 1)$
 - lo forzamos Gaussiano

$$\Pr(\mathbf{h} | \mathbf{x}_n; \mathbf{W}_f) = N(\mathbf{h}; \mu_n(\mathbf{x}_n; \mathbf{W}_f), \sigma_n(\mathbf{x}_n; \mathbf{W}_f) \mathbf{I})$$

- La media y la varianza los obtenemos con la red neuronal dado $\mathbf{X}$ y $\mathbf{W}_f$

VAE likelihood aproximación

Aproximamos la integral

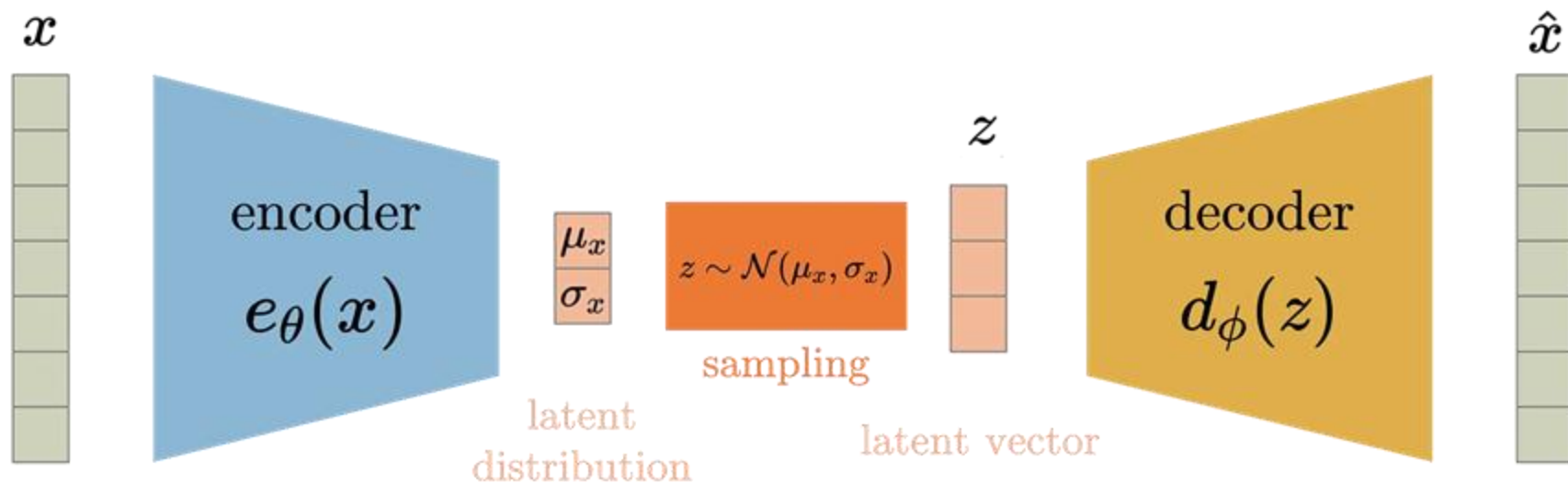
$$\begin{aligned} \Pr(\mathbf{x}_n; \mathbf{W}_f, \mathbf{W}_g) \\ = \int_{\mathbf{h}} \Pr(\mathbf{x}_n \mid \mathbf{h}; \mathbf{W}_g) N(\mathbf{h}; \mu_n(\mathbf{x}_n; \mathbf{W}_f), \sigma_n(\mathbf{x}_n; \mathbf{W}_f) \mathbf{I}) \end{aligned}$$

Con un único sample

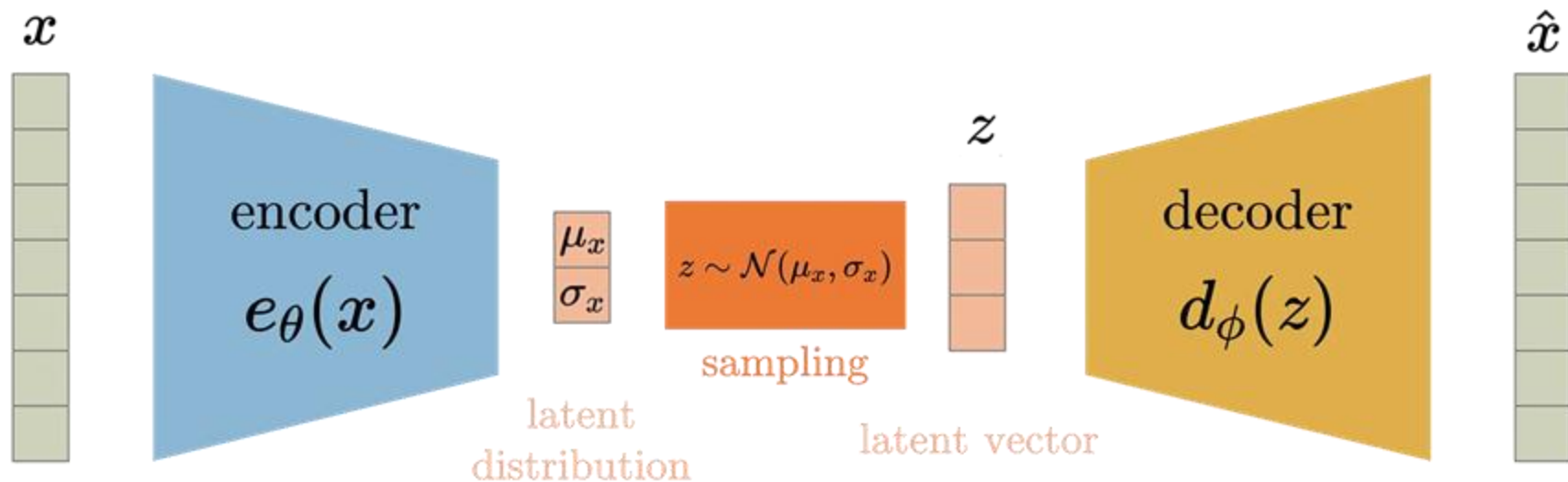
$$\begin{aligned} \Pr(\mathbf{x}_n; \mathbf{W}_f, \mathbf{W}_g) &\approx \Pr(\mathbf{x}_n \mid \mathbf{h}_n; \mathbf{W}_g) \\ \text{where } \mathbf{h}_n &\sim N(\mathbf{h}; \mu_n(\mathbf{x}_n; \mathbf{W}_f), \sigma_n(\mathbf{x}_n; \mathbf{W}_f) \mathbf{I}) \end{aligned}$$

Pero después de muchas iteraciones en la red va a funcionar

¿Cómo obtenemos la media y la varianza?

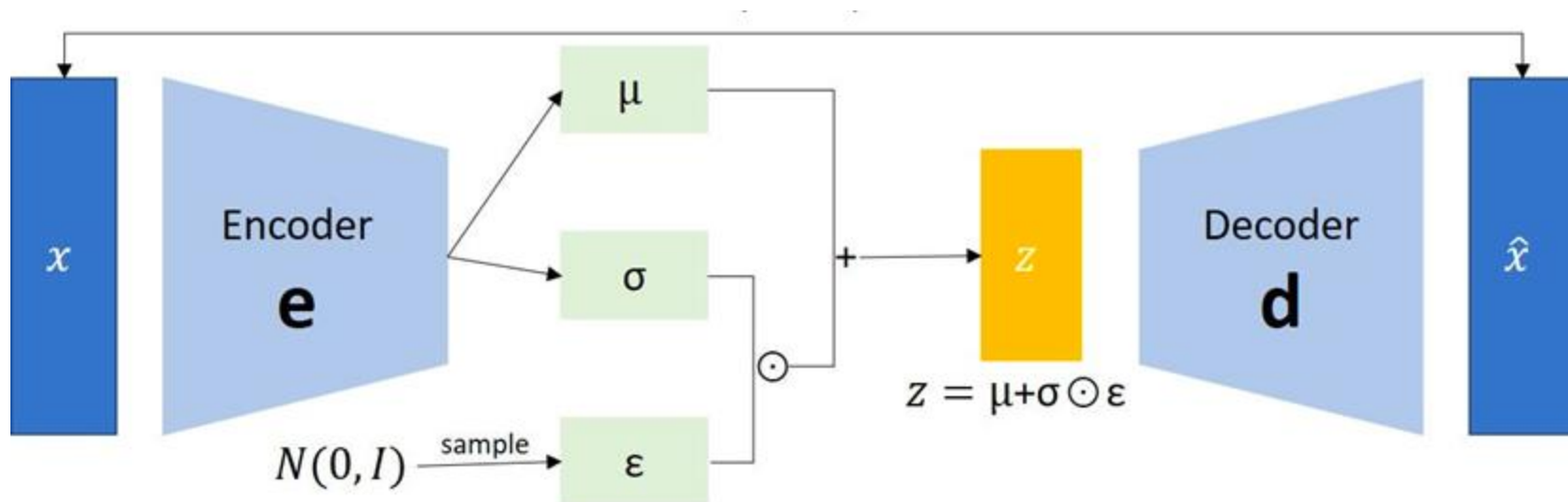


¿Cómo obtenemos la media y la varianza?



Random dentro de la red $\rightarrow$ ¿¿backpropagation??

Truco de re parametrización



VAE para generación

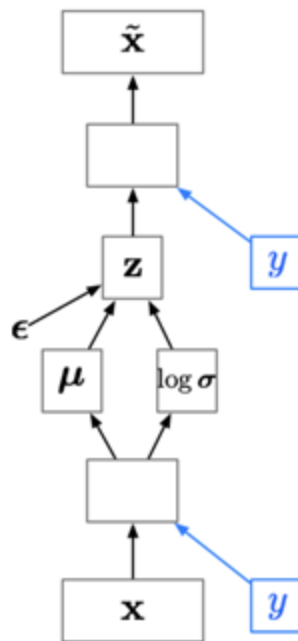
- VAE es un autoencoder, pero también nos permite generar
- La generación sólo requiere de una pasada forward
- Podemos fitear espacios latentes de bajas dimensiones y hacer cosas interesantes en ellos



VAEs condicional a clases

- Un VAE condicionado por clases proporciona las etiquetas tanto al codificador como al decodificador.
- Dado que el código latente z ya no tiene que modelar la categoría de la imagen, puede centrarse en modelar las características estilísticas.
 - nos permite desenredar el estilo y el contenido.

Nota: el desenredamiento todavía es un arte oscuro



Manipulación del espacio latente

Variando dos dimensiones latentes (es decir, dimensiones de z) mientras mantenemos y fijo, podemos visualizar el espacio latente.



Manipulación del espacio latente

Variando la etiqueta y mientras mantenemos z fijo, podemos resolver analogías de imágenes.



Manipulación del espacio latente

Se pueden obtener resultados interesantes al interpolar entre dos vectores en el espacio latente



Agenda

1. AutoEncoder
2. Variational Autoencoders
- 3. Generative Adversarial Networks**
4. Bibliografía



Generative Adversarial Networks

By Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu,
David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio

Abstract

Generative adversarial networks are a kind of artificial intelligence algorithm designed to solve the *generative modeling* problem. The goal of a generative model is to study a collection of training examples and learn the probability distribution that generated them. Generative Adversarial Networks (GANs) are then able to generate more examples from the estimated probability distribution. Generative models based on deep learning are common, but GANs are among the most successful generative models (especially in terms of their ability to generate realistic high-resolution images). GANs have been successfully applied to a wide variety of tasks (mostly in research settings) but continue to present unique challenges and research opportunities because they are based on game theory while most other approaches to generative modeling are based on optimization.

1. INTRODUCTION

Most current approaches to developing artificial intelligence are based primarily on machine learning. The most widely used and successful form of machine learning to date is supervised learning. Supervised learning algorithms are given a dataset of pairs of example inputs and example outputs. They learn to associate each input with each output and thus learning a mapping from input to output examples. The input examples are typically complicated data objects like images, natural language sentences, or audio waveforms, while the output examples are often relatively simple. The most common kind of supervised learning is classification, where the output is just an integer code identifying a specific category (a photo might be recognized as coming from category 0 containing cats, or category 1 containing dogs, etc.).

Supervised learning is often able to achieve greater than human accuracy after the training process is complete, and thus has been integrated into many products and services.

applications of GANs and identify core research problems related to convergence in games necessary to make GANs a reliable technology.

2. GENERATIVE MODELING

The goal of supervised learning is relatively straightforward to specify, and all supervised learning algorithms have essentially the same goal: learn to accurately associate new input examples with the correct outputs. For instance, an object recognition algorithm may associate a photo of a dog with some kind of DOG category identifier.

Unsupervised learning is a less clearly defined branch of machine learning, with many different unsupervised learning algorithms pursuing many different goals. Broadly speaking, the goal of unsupervised learning is to learn something useful by examining a dataset containing unlabeled input examples. Clustering and dimensionality reduction are common examples of unsupervised learning.

Another approach to unsupervised learning is generative modeling. In generative modeling, training examples $\mathbf{x}$ are drawn from an unknown distribution $p_{\text{data}}(\mathbf{x})$. The goal of a generative modeling algorithm is to learn a $p_{\text{model}}(\mathbf{x})$ that approximates $p_{\text{data}}(\mathbf{x})$ as closely as possible.

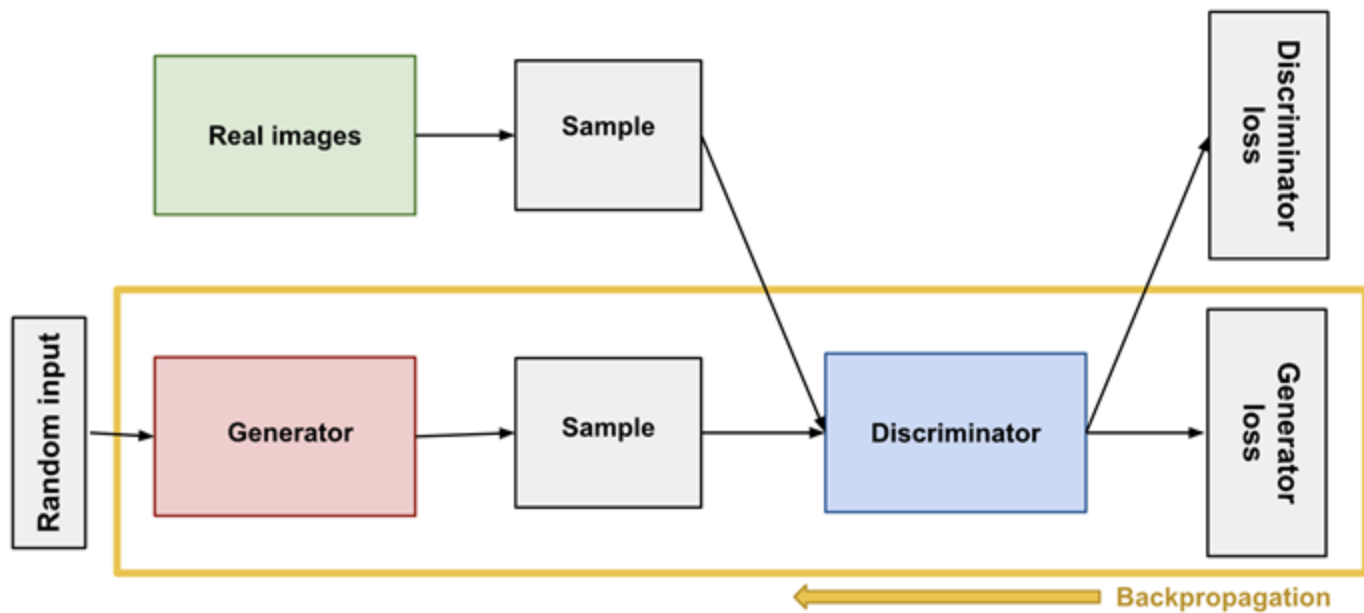
A straightforward way to learn an approximation of p_{data} is to explicitly write a function $p_{\text{model}}(\mathbf{x}; \theta)$ controlled by parameters θ and search for the value of the parameters that makes p_{data} and p_{model} as similar as possible. In particular, the most popular approach to generative modeling is probably *maximum likelihood estimation*, consisting of minimizing the Kullback-Leibler divergence between p_{data} and p_{model} . The common approach of estimating the mean parameter of a Gaussian distribution by taking the mean of a set of observations is one example of maximum likelihood estimation. This approach based on explicit density functions is illustrated in Figure 1.

Explicit density modeling has worked well for traditional statistics, using simple functional forms of probability distributions, usually applied to small numbers of variables.

Generative Adversarial Networks (GANs)

- Basado en teoría de juego
- Dos redes
 1. Generador $g(z; W_g) \rightarrow x$
 2. Discriminador $d(x; W_d) \rightarrow P(x \text{ es real})$





Generative Adversarial Networks (GANs)

Función de objetivo

$$\begin{aligned} \min_{W_g} \max_{W_d} \sum_n \log \Pr(x_n \text{ is real} ; W_d) + \log \Pr(g(\mathbf{z}_n; \mathbf{W}_g) \text{ is fake}; \mathbf{W}_d) \\ \equiv \min_{W_g} \max_{W_d} \sum_n \log d(x_n; W_d) + \log(1 - d(g(\mathbf{z}_n; \mathbf{W}_g); \mathbf{W}_d)) \end{aligned}$$

Entrenamiento

Repeat until convergence

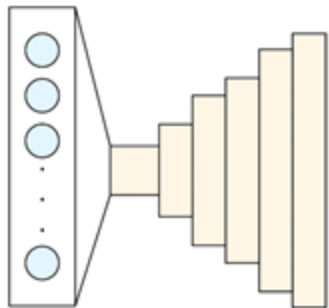
- for 1 to k
 - a. samplear z de $P(z)$
 - b. samplear x del set de train
 - c. actualizar el discriminador con **ascenso** del gradiente estocástico

$$\nabla_{W_d} \left(\frac{1}{N} \sum_{n=1}^N [\log d(\mathbf{x}_n; \mathbf{W}_d) + \log(1 - d(g(\mathbf{z}_n; \mathbf{W}_g); \mathbf{W}_d))] \right)$$

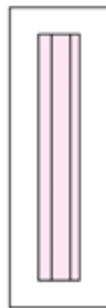
- samplear x de $P(z)$
- actualizar el generador con descenso de gradiente estocástico

$$\nabla_{W_g} \left(\frac{1}{N} \sum_{n=1}^N \log(1 - d(g(\mathbf{z}_n; \mathbf{W}_g); \mathbf{W}_d)) \right)$$

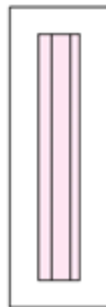
Latent Space



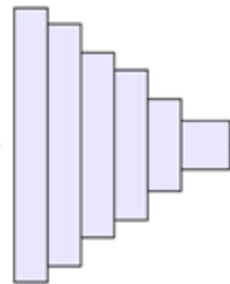
Generator(G)



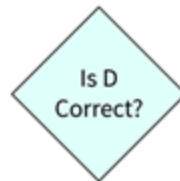
Generated
Fake Samples



Real
Samples



Discriminator(D)



Is D
Correct?

Entrenamiento

- Con redes lo suficientemente expresivas
 - $P(x | z, w_g) \rightarrow$ se asemeja a la verdadera distribución de los datos
 - $P(x \text{ es real} | W_d) \rightarrow 0.5$ (para datos reales y falsos)
- Problemas en la práctica
 - Desbalance
 - una red tiende a dominar a la otra
 - Convergencia local

Bibliografía

- Variational autoencoders (VAEs):
 - <https://arxiv.org/pdf/1312.6114.pdf>
- Generative adversarial networks (GANs)
 - <https://arxiv.org/pdf/1312.6114.pdf>